

FINAL PRACTICE!

Problem Solving with Computers-II

Link to class handout:

<https://bit.ly/one-problem-to-rule-them-all>

The image shows the C++ logo in a large, blue, 3D-style font. Below the logo is a snippet of C++ code in a monospaced font, with some words highlighted in color (pink for #include, using, int, main; and green for cout). The code is:

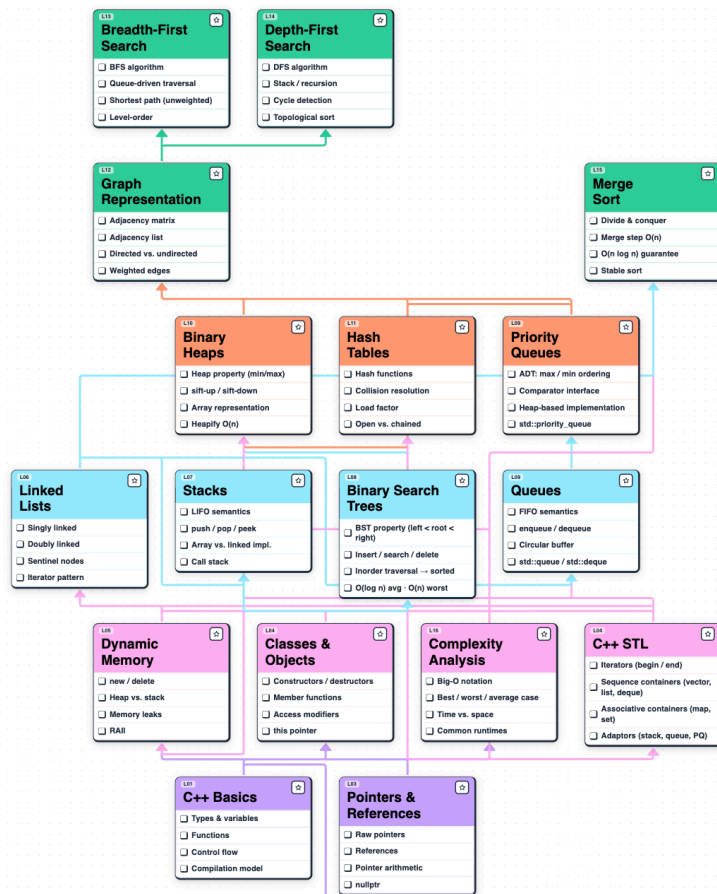
```
#include <iostream>
using namespace std;

int main() {
    cout<<"Hola Facebook\n";
    return 0;
}
```

I can deal with pressure, and deadlines.



Please log into Caliber → Mock
Interview Scheduler → Fill the End of Quarter Survey
5 mins



One problem to rule them all!

Problem (LP04): Longest Consecutive Sequence

Description: Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

Try solving it in as many different ways as you can, then pick the best!

Ex: `nums = [100, 4, 200, 1, 3, 2]`

output = 4

`nums = [1, 0, 1, 2]`

output : 3

One problem to rule them all!

Problem (LP04): Longest Consecutive Sequence

Description: Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

Try solving it in as many different ways as you can, then pick the best!

Ex: `nums = [100, 4, 200, 1, 3, 2]`

`nums = [1, 0, 1, 2]`



Approach 1 : Sort the vector + iterate to find the longest streak.

nums = [100, 4, 200, 1, 3, 2]

nums = [1, 0, 1, 2]

Step 1 -> Sort

$[\overset{0}{\underline{1}}, \overset{1}{\underline{2}}, \overset{2}{\underline{3}}, \overset{3}{\underline{4}}, \overset{4}{\underline{100}}, \overset{5}{\underline{200}}]$

nums[i] + 1 == nums[i+1] ?

1 + 1 == 2 ? T

2 + 1 == 3 ? T

3 + 1 == 4 ? T

4 + 1 == 100 ? F

100 + 1 == 200 ? F

seq len = 1

seq len ++ 2

" 3

" 4

seq len 1

1

result = 0

max(result, seq len)

= 4

= 4

1
0
1
2
3
4

lcs (nums):

sort nums $\rightarrow$ nums

seqlen = 1

result = 0

for $i = 0$ to $n-2$

$\rightarrow$ # handle the duplicate case.

if $\text{nums}[i] + 1 == \text{nums}[i+1]$:

seqlen ++

else

result = max(result, seqlen)

seqlen = 1

return result

nums: [0, 0, 1, 2, 3, 4, 5, 6, 7, 8]

output = 9

my answer = 0

Approach 2 : Use a DS that maintains order BST $\rightarrow$ Set or Partial order PQ

nums = [100, 4, 200, 1, 3, 2]

nums = [1, 0, 1, 2]

① Put all the keys in a set (BST) (S)

② Inorder traversal of the set

num : S

Does num+1 exist in S ?

seqLen=1

result=0 Set (S)

s.count(num+1)

1

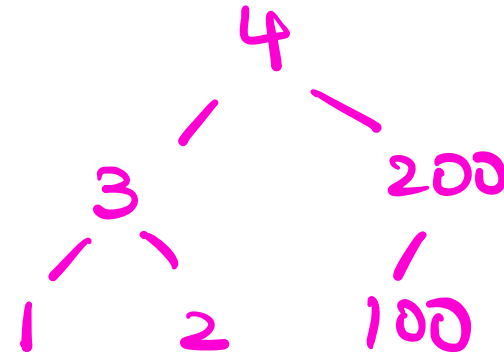
2

3

4

100

200



Approach 3 $\rightarrow$ No sorting, no sorted DS !

nums = [100, 4, 200, 1, 3, 2]

H = { 3, 100, 1, 2, 200, 4 }

num: H

3

$\rightarrow$

100

1

H. count (num+1)

4 ✓

5 X

101 X

2 ✓

3 ✓

4 ✓

5 X

seq = 2

seq = 1

seq 2 4

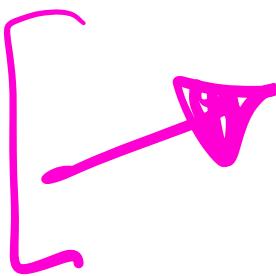
Approach 4 ? Same as before : hop through unordered_map (hash table)
 → use a recursive approach

nums = [9, 7, 6, 8, 4, 5]

Step 1: $M: \{ 9:0, 7:0, 6:0, 8:0, 4:0, 5:0 \}$

Step 2: Recursively hop through the map to compute lcs

Recursive def:

$$\begin{aligned}
 \text{lcs}(\underline{M}, \text{key}) &= \underline{0} \quad \text{if } \text{key} \notin M \\
 &= 1 + \text{lcs}(M, \text{key} + 1), \text{ otherwise}
 \end{aligned}$$


Step 1: $M: \{ \underline{9}:0, 7:0, 6:0, \underline{8}:0, 4:0, 5:0 \}$
 $\quad \quad \quad \underline{1} \quad 1+2 \quad 1+1$
 $\quad \quad \quad \quad \quad = 3 \quad \quad = 2$

longest seq (nums):

Put all elements in nums in the map M

$M[num] = 0$

maxlen = 0

→ for each key k in M :

$maxlen = \max(maxlen, lcs(M, k))$

return maxlen

Optimized version)

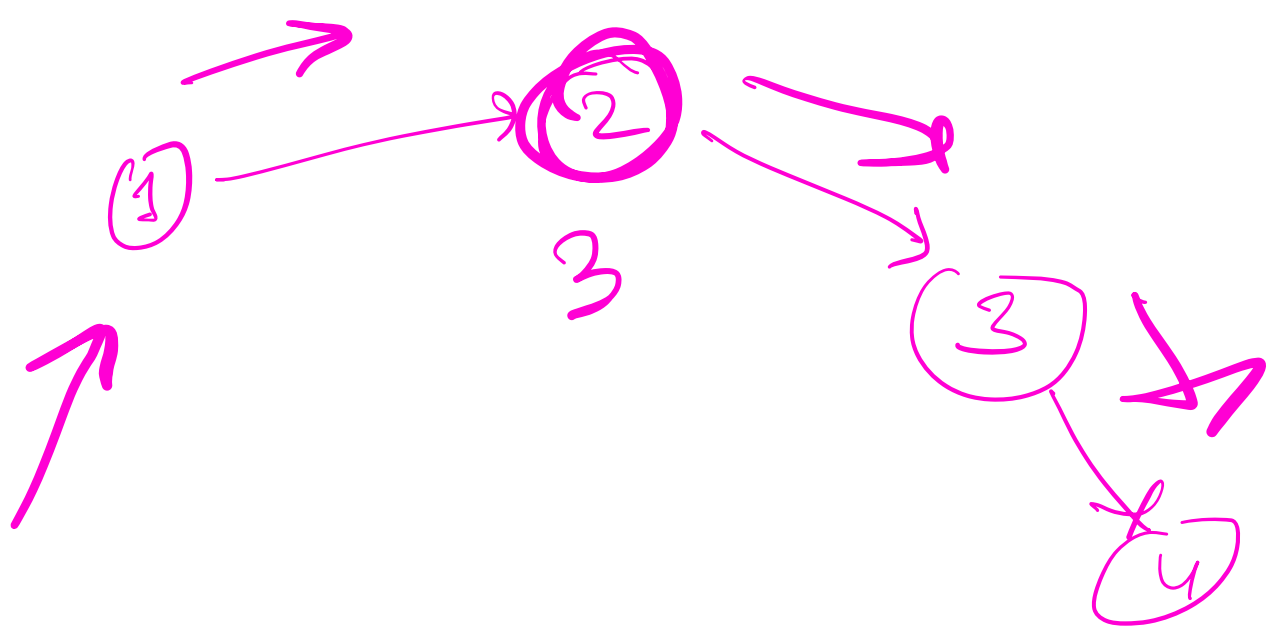
$lcs(M, key)$:

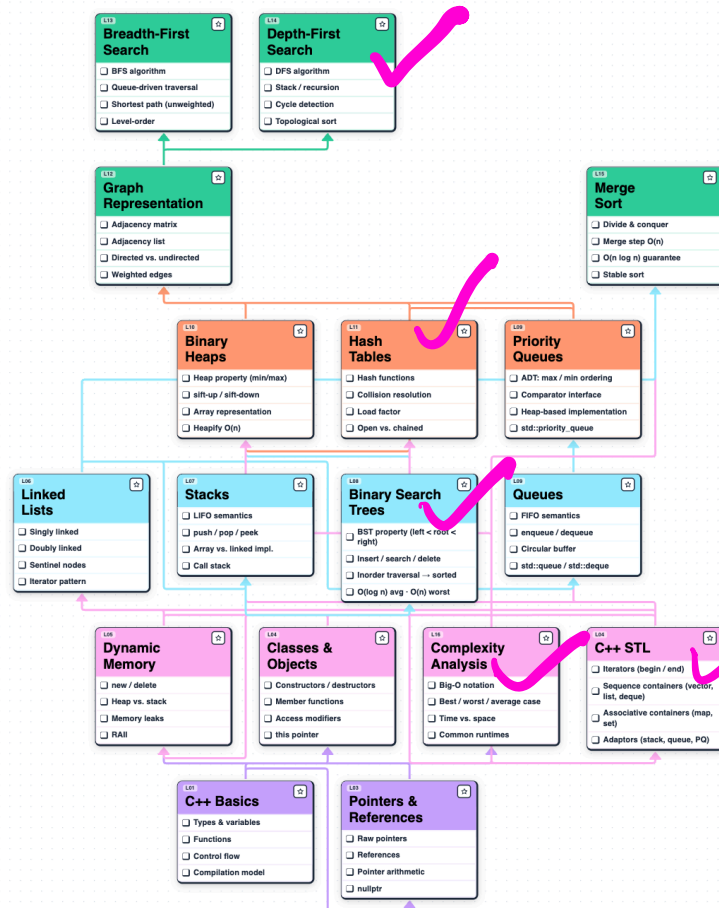
if key is not in M :
 return 0

if $M[key] > 0$: return $M[key]$

$M[key] = 1 + lcs(M, key + 1)$

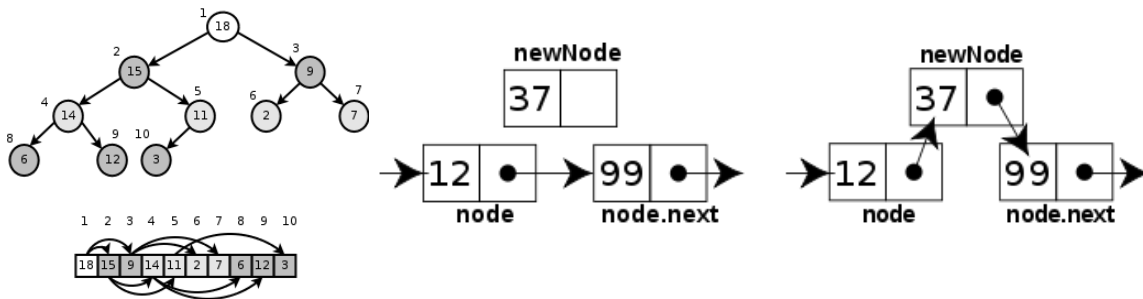
return $M[key]$





Resources for the Final Exam

- Leet code problems categorized to help you focus on the most relevant ones.
 - Those marked as “Optional challenge” won’t be on the final
- Code from lectures: <https://github.com/ucsb-cs24-s26/cs24-s26-lectures>
- Practice final exam on Canvas
- Tool to visualize data structures: <https://visualgo.net/>
- If you missed your Mock interview, complete during section tomorrow.



INSERTION-SORT(A)

```

1  for  $j = 2$  to  $A.length$ 
2     $key = A[j]$ 
3    // Insert  $A[j]$  into the sorted
   sequence  $A[1..j-1]$ .
4     $i = j - 1$ 
5    while  $i > 0$  and  $A[i] > key$ 
6       $A[i + 1] = A[i]$ 
7       $i = i - 1$ 
8     $A[i + 1] = key$ 
  
```

cost	times
c_1	n
c_2	$n - 1$
0	$n - 1$
c_4	$n - 1$
c_5	$\sum_{j=2}^n t_j$
c_6	$\sum_{j=2}^n (t_j - 1)$
c_7	$\sum_{j=2}^n (t_j - 1)$
c_8	$n - 1$

Data Structures and C++

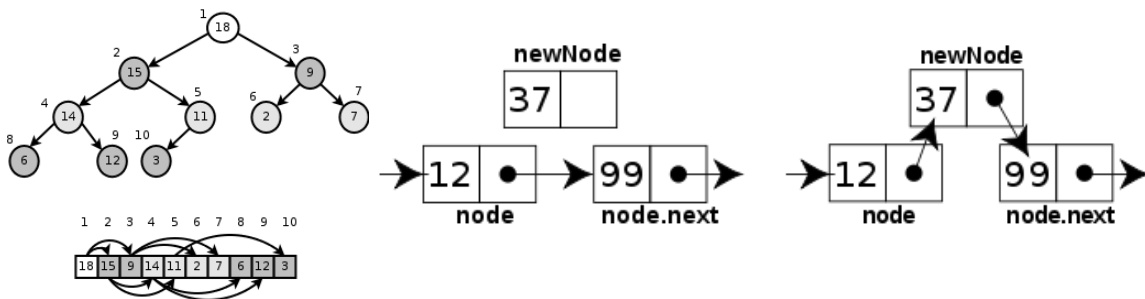
Complexity Analysis

CS24: Learning goals

Correct coding, clear thinking, no AI shortcuts

- Design and implement **larger programs** that **run fast**
- Organize **data** in programs using **data structures**
- **Analyze** the **complexity** of your programs
- Prep for **technical interviews**

Grow confident in your problem solving skills!



INSERTION-SORT(A)

```

1  for  $j = 2$  to  $A.length$ 
2     $key = A[j]$ 
3    // Insert  $A[j]$  into the sorted
   sequence  $A[1..j-1]$ .
4     $i = j - 1$ 
5    while  $i > 0$  and  $A[i] > key$ 
6       $A[i + 1] = A[i]$ 
7       $i = i - 1$ 
8     $A[i + 1] = key$ 
  
```

cost	times
c_1	n
c_2	$n - 1$
0	$n - 1$
c_4	$n - 1$
c_5	$\sum_{j=2}^n t_j$
c_6	$\sum_{j=2}^n (t_j - 1)$
c_7	$\sum_{j=2}^n (t_j - 1)$
c_8	$n - 1$

Data Structures and C++

Complexity Analysis

Thank you and all the best !



Please take a moment to fill the course evaluations!



PROBLEM SOLVING II

Student-FO

<https://go.blueja.io/IKRcDtvefUOISGeejnmLjw>

To access the evaluation, scan this QR code with your mobile phone.